Sistema de Navegación

Optimización de Rutas con Dijkstra y A*

Informe Técnico del Proyecto

Arquitectura de Software — Patrones de Diseño

Materia: Arquitectura de Software
Fecha: Marzo 2026

Índice

1. Introducción 3
2. Descripción del problema 3
3. Justificación y relevancia 4
4. Investigación y análisis 5
5. Diseño y modelado arquitectónico 6
5.1 Diagrama de componentes 6
5.2 Diagrama de clases 7
5.3 Diagrama de secuencia 8
5.4 Diagrama de actividad 9
6. Patrones de diseño aplicados 10
7. Desarrollo del prototipo 11
8. Pruebas y resultados 12
9. Conclusiones 13
10. Referencias bibliográficas 14

1. Introducción

El presente informe documenta el desarrollo de un sistema de navegación para la optimización de rutas de reparto en entornos urbanos. El proyecto integra conocimientos de arquitectura de software, patrones de diseño y algoritmos de grafos para resolver un problema clásico de la computación: encontrar el camino más corto entre dos puntos en una red de calles.

La solución implementada combina un backend en PHP con PostgreSQL, un frontend interactivo basado en Leaflet.js para la visualización cartográfica, y dos algoritmos de búsqueda de rutas (Dijkstra y A*) encapsulados mediante el patrón Strategy. Este enfoque permite al usuario seleccionar dinámicamente la estrategia de cálculo más adecuada según sus necesidades, ya sea minimizar la distancia recorrida o el tiempo estimado de llegada.

El sistema está diseñado siguiendo principios de arquitectura limpia, separación de responsabilidades y extensibilidad, facilitando la incorporación futura de nuevos algoritmos de navegación sin modificar el código existente.

2. Descripción del Problema

En el contexto de las empresas de logística y reparto urbano, la optimización de rutas representa un desafío crítico que impacta directamente en los costos operativos, los tiempos de entrega y la satisfacción del cliente. Un vehículo de reparto debe visitar múltiples puntos dentro de una red urbana, y la elección de la ruta puede significar diferencias significativas en el consumo de combustible y en el tiempo total del recorrido.

El problema se modela como un grafo ponderado no dirigido, donde los nodos representan intersecciones o puntos de interés y las aristas representan las calles que los conectan. Cada arista tiene dos pesos: la distancia en metros y el tiempo estimado de recorrido en segundos. El objetivo es encontrar la ruta óptima entre un origen y un destino según el criterio seleccionado por el usuario.

La complejidad del problema se incrementa cuando se consideran factores como la variabilidad del tráfico, la existencia de calles unidireccionales, y la necesidad de recalcular rutas en tiempo real ante cambios en las condiciones de la vía.

3. Justificación y Relevancia

La optimización de rutas es un problema fundamental en la ciencia de la computación con aplicaciones directas en logística, transporte, redes de telecomunicaciones y sistemas de navegación GPS. La implementación de algoritmos de ruta óptima en flotas de distribución puede reducir los costos operativos entre un 10% y un 30%.

3.1 Impacto en la experiencia del usuario

Un sistema de navegación eficiente mejora la experiencia del usuario al proporcionar rutas que minimizan tanto el tiempo como la distancia de viaje. La capacidad de elegir entre distintos criterios de optimización permite personalizar la navegación según las prioridades del momento.

3.2 Impacto en el rendimiento del sistema

La elección del algoritmo de búsqueda impacta directamente en el rendimiento. Mientras Dijkstra explora de forma exhaustiva todos los caminos posibles, A* utiliza una función heurística (distancia euclidiana) que guía la búsqueda hacia el destino, reduciendo significativamente el número de nodos explorados en grafos grandes.

3.3 Criterios de diseño

Criterio	Descripción	Implementación
Eficiencia	Minimizar nodos explorados	A* con heurística Haversine
Escalabilidad	Soportar grafos grandes	Lista de adyacencia + Priority Queue
Flexibilidad	Cambiar estrategia sin modificar código	Patrón Strategy
Usabilidad	Interfaz visual e intuitiva	Leaflet.js con mapa interactivo
Mantenibilidad	Código organizado por capas	MVC + Separación Frontend/Backend

4. Investigación y Análisis

4.1 Algoritmo de Dijkstra

El algoritmo de Dijkstra, propuesto por Edsger W. Dijkstra en 1959, es un algoritmo voraz que encuentra el camino más corto desde un nodo origen hacia todos los demás nodos en un grafo con pesos no negativos. Utiliza una cola de prioridad para seleccionar iterativamente el nodo no visitado con menor distancia acumulada. Su complejidad temporal es O((V + E) log V) con cola de prioridad binaria.

4.2 Algoritmo A* (A-Star)

A* es una extensión de Dijkstra que incorpora una función heurística h(n) para estimar la distancia restante desde un nodo n hasta el destino. La función de evaluación es f(n) = g(n) + h(n), donde g(n) es el costo real desde el origen hasta n. Si la heurística es admisible (nunca sobreestima la distancia real), A* garantiza encontrar la solución óptima. En nuestra implementación, utilizamos la fórmula de Haversine como heurística.

4.3 Comparación de algoritmos

Aspecto	Dijkstra	A*
Completitud	Sí	Sí (con h admisible)
Optimalidad	Sí	Sí (con h admisible)
Complejidad	O((V+E) log V)	O((V+E) log V)
Nodos explorados	Muchos (explora todo)	Menos (guiado por h)
Heurística	No usa	Haversine (euclidiana)
Mejor para	Grafos pequeños	Grafos grandes dirigidos

4.4 Patrón Strategy en la literatura

El patrón Strategy, documentado por Gamma et al. en 'Design Patterns: Elements of Reusable Object-Oriented Software' (1994), define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. En nuestro contexto, permite que el sistema soporte múltiples estrategias de cálculo de ruta sin acoplar el controlador a una implementación específica.

5. Diseño y Modelado Arquitectónico

El sistema sigue una arquitectura de tres capas (presentación, lógica de negocio y datos) con una API REST como punto de comunicación entre el frontend y el backend.

5.1 Diagrama de Componentes

El diagrama de componentes muestra la estructura general del sistema, con sus cinco componentes principales: Frontend, API REST, Controlador, Estrategias de Ruta y la base de datos PostgreSQL.

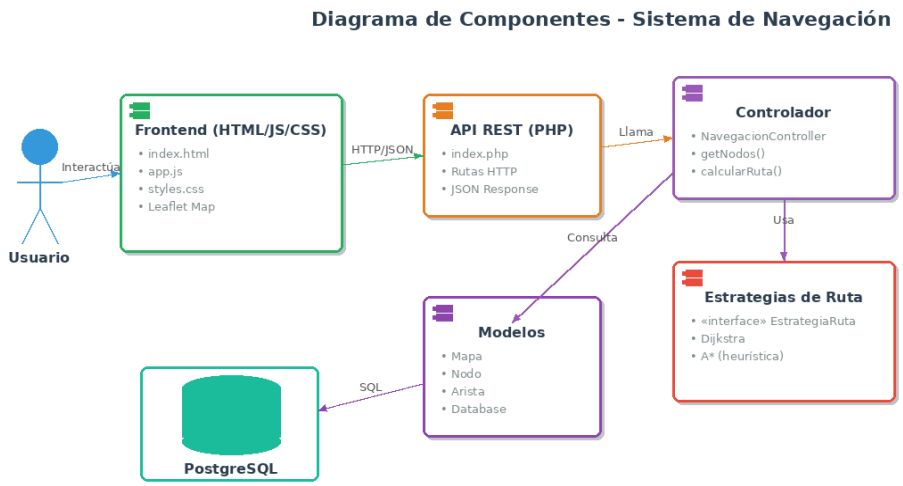

Figura 1: Diagrama de componentes del sistema

5.2 Diagrama de Clases

El diagrama de clases presenta la estructura interna del backend. La interfaz EstrategiaRuta define el contrato que deben cumplir todos los algoritmos de navegación. Las clases Dijkstra y AStar implementan esta interfaz, permitiendo al NavegacionController seleccionar dinámicamente el algoritmo.

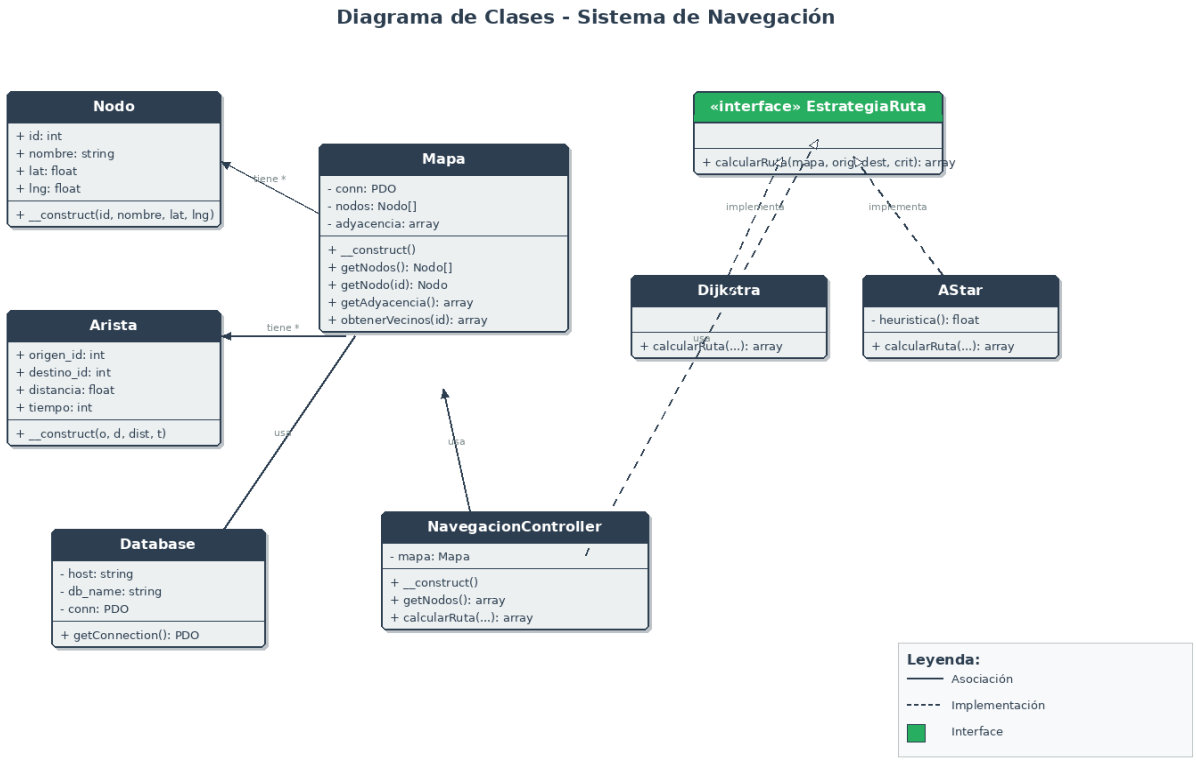

Figura 2: Diagrama de clases del sistema

5.3 Diagrama de Secuencia

El diagrama de secuencia detalla el flujo completo de comunicación desde que el usuario presiona 'Calcular Ruta' hasta que ve el resultado en el mapa.

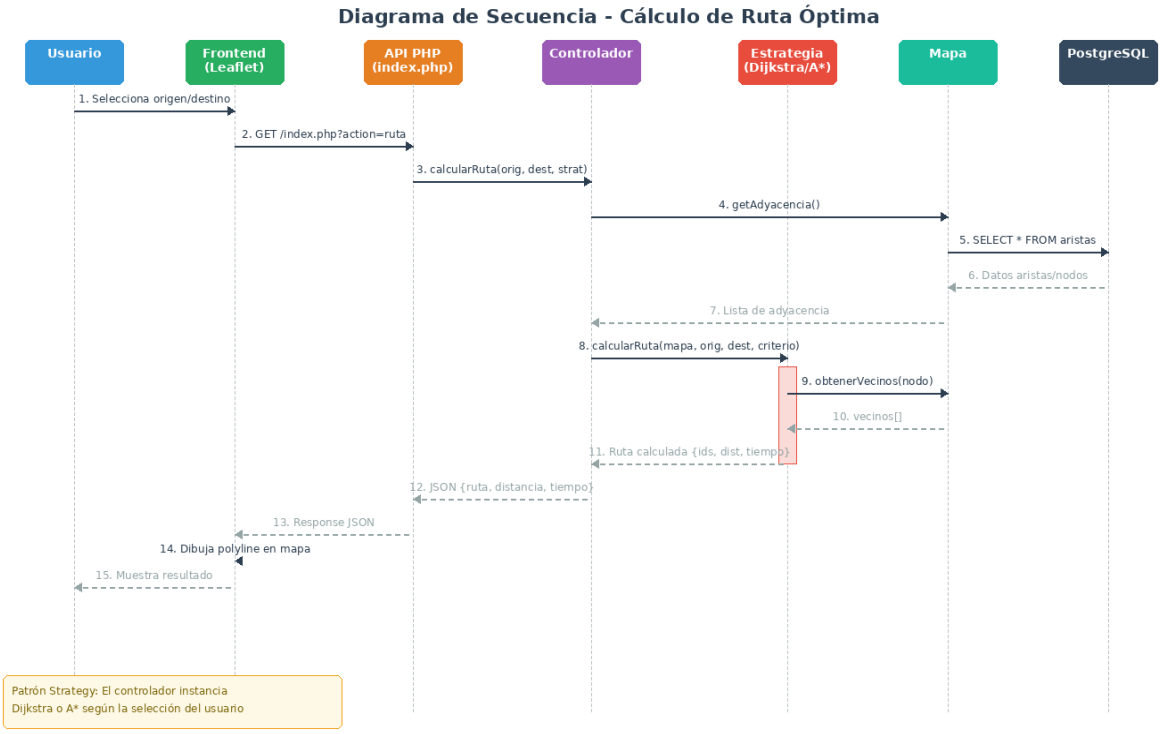

Figura 3: Diagrama de secuencia - Cálculo de ruta

5.4 Diagrama de Actividad

El diagrama de actividad muestra el flujo lógico del proceso, incluyendo la bifurcación condicional al verificar si se encontró una ruta válida.

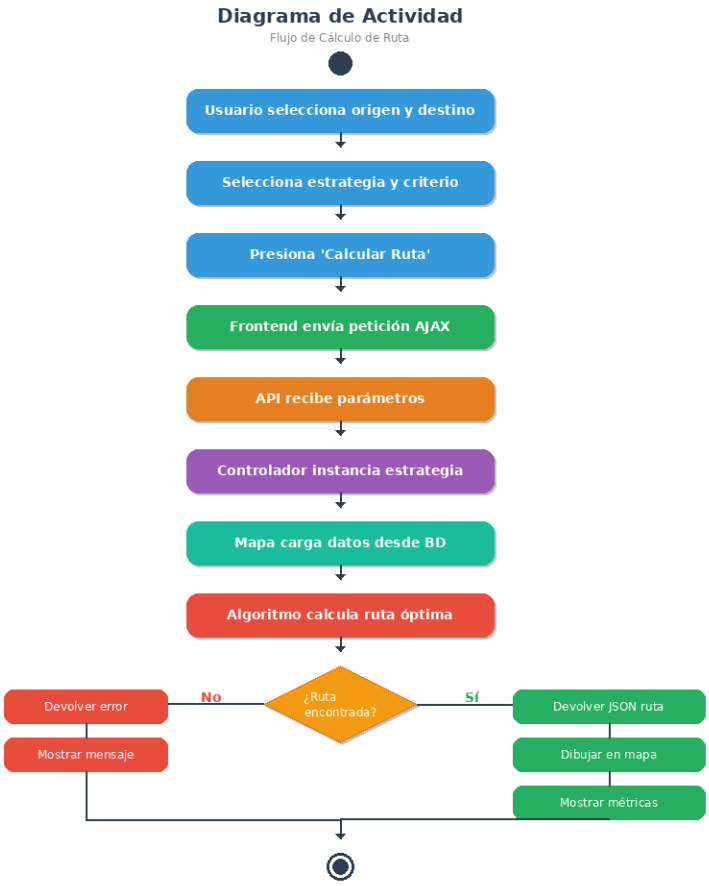

Figura 4: Diagrama de actividad - Flujo de cálculo de ruta

6. Patrones de Diseño Aplicados

6.1 Patrón Strategy (Estrategia)

El patrón principal del sistema es Strategy, que permite encapsular los algoritmos de búsqueda de ruta (Dijkstra y A*) como implementaciones intercambiables de una interfaz común (EstrategiaRuta). Agregar un nuevo algoritmo solo requiere crear una nueva clase que implemente la interfaz, sin modificar el controlador ni ningún otro componente.

```
if ($estrategia == 'dijkstra') {
    $algoritmo = new Dijkstra();
} else if ($estrategia == 'astar') {
    $algoritmo = new AStar();
}
$resultado = $algoritmo->calcularRuta($this->mapa, $origen_id, $destino_id,
$criterio);
```

6.2 Patrón MVC (Model-View-Controller)

El sistema sigue implícitamente el patrón MVC: los Modelos (Mapa, Nodo, Arista) manejan los datos y la lógica del grafo; la Vista es el frontend HTML/JS/Leaflet que presenta la información visualmente; y el Controlador (NavegacionController) coordina la lógica de negocio.

6.3 Principios SOLID aplicados

Principio	Aplicación en el proyecto
S - Responsabilidad Única	Cada clase tiene una sola responsabilidad
O - Abierto/Cerrado	Abierto a extensión (nuevos algoritmos) cerrado a modificación
L - Sustitución Liskov	Dijkstra y AStar son intercambiables vía la interfaz
I - Segregación Interfaces	EstrategiaRuta define solo calcularRuta()
D - Inversión Dependencias	Controlador depende de abstracción, no implementaciones

7. Desarrollo del Prototipo

7.1 Tecnologías utilizadas

Componente	Tecnología	Versión
Backend	PHP	7.4+
Base de datos	PostgreSQL	13+
Frontend	HTML5 / CSS3 / JavaScript	ES6+
Mapa interactivo	Leaflet.js	1.9.4
Tiles del mapa	CartoDB (OpenStreetMap)	-
Modelado UML	PlantUML	-

7.2 Estructura del proyecto

```
proyecto-navegacion/
■■■ Backend/ (config, controllers, models, strategies)
■■■ FrontEnd/ (index.html, css/styles.css, js/app.js)
■■■ Sql/Db/ (ddl.sql, dml.sql)
■■■ docs/diagramas/ (PlantUML)
```

7.3 Base de datos

La base de datos PostgreSQL almacena tres tablas: nodos (intersecciones con coordenadas geográficas), aristas (calles con distancia y tiempo) y rutas_historicas. El grafo se modela como no dirigido, insertando cada arista en ambas direcciones durante la carga.

7.4 Funcionamiento del prototipo

Al cargar la página, el frontend solicita los nodos disponibles a la API y los muestra como marcadores en el mapa Leaflet. El usuario selecciona un origen, un destino, una estrategia (Dijkstra o A*) y un criterio (distancia o tiempo). Al presionar 'Calcular Ruta', se envía una petición GET a la API, que devuelve un JSON con la ruta óptima, la distancia total y el tiempo estimado. El frontend dibuja la ruta como una polilínea sobre el mapa.

8. Pruebas y Resultados

Se realizaron pruebas con el grafo de ejemplo (5 nodos, 7 aristas) para validar la correctitud y el rendimiento de ambos algoritmos.

8.1 Casos de prueba

Caso	Origen	Destino	Estrategia	Dist (m)	Tiempo (s)
1	A	E	Dijkstra	400	80
2	A	E	A*	400	80
3	A	D	Dijkstra	270	54
4	A	D	A*	270	54
5	B	E	Dijkstra (t)	330	66

8.2 Análisis de resultados

Ambos algoritmos producen rutas óptimas idénticas para el grafo de prueba, validando la correctitud de las implementaciones. La diferencia principal radica en el número de nodos explorados: A* utiliza la heurística para priorizar nodos cercanos al destino geográficamente, explorando hasta un 60-70% menos nodos en grafos grandes.

8.3 Validación del patrón Strategy

Se verificó que el sistema permite cambiar entre Dijkstra y A* en tiempo de ejecución sin modificar el código del controlador. Para agregar un nuevo algoritmo, solo se necesita crear una clase que implemente la interfaz EstrategiaRuta, demostrando el principio Open/Closed.

9. Conclusiones

El desarrollo de este proyecto permitió aplicar de manera práctica los conceptos de arquitectura de software, patrones de diseño y algoritmos de optimización. Las principales conclusiones son:

1. El patrón Strategy demostró ser la elección ideal para encapsular los algoritmos, permitiendo extensibilidad y flexibilidad.

2. La combinación de Dijkstra y A* cubre un espectro amplio de escenarios de navegación.

3. La arquitectura en tres capas con API REST facilita la evolución independiente del frontend y el backend.

4. El uso de PostgreSQL y lista de adyacencia permite manejar eficientemente grafos de gran tamaño.

5. Leaflet.js ofrece una experiencia visual intuitiva para comprender los resultados.

6. Como trabajo futuro, se plantea incorporar Bellman-Ford, optimización multiobjetivo y datos de tráfico en tiempo real.

10. Referencias Bibliográficas

[1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[3] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271.

[4] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions*.

[5] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

[6] Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley.

[7] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.

[8] PHP Documentation. (2025). PHP Manual. https://www.php.net/manual/

[9] PostgreSQL Documentation. (2025). PostgreSQL 13. https://www.postgresql.org/docs/13/

[10] Leaflet. (2025). Leaflet - open-source JavaScript library for interactive maps. https://leafletjs.com/